

C++&Qt Day3 hw3 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 파일 구조
3. 코드 구조
4. 실행결과
5. 어려웠던 점

1. 프로젝트 개요

이 프로젝트는 옛날 폴더폰의 문자 입력 방식을 흉내낸 키패드 프로그램이다. 숫자 버튼 하나에 알파벳 여러 개가 들어 있어서, 같은 버튼을 연속으로 누르면 글자가 바뀌고 잠시 기다리면 그 글자로 확정된다. 엔터를 누르면 지금까지 입력한 문장이 output.txt에 저장된다.

앞선 과제들에서는 버튼을 누르면 바로 결과가 나오는 방식이었는데, 이번에는 같은 버튼을 눌러도 상황에 따라 다른 동작을 해야 한다는 점이 다르다. 방금 전에 무슨 키를 눌렀는지, 몇 번째로 누른 건지를 기억하고 있어야 하기 때문에 상태를 저장하는 변수들이 필요했다. 그래서 이번 과제는 QTimer를 이용해 시간을 기준으로 동작을 나누는 방법과, 상태 변수를 두어 프로그램의 동작을 관리하는 방법을 학습하는데에 초점을 두었다.

또한 버튼이 14개나 되기 때문에 connect를 하나씩 다 적으면 코드가 너무 길어진다. 그래서 숫자 버튼들은 배열에 담아 반복문으로 한 번에 연결하는 방법을 사용해 보았다.

2. 파일 구조

Project_name

```
|—— CMakeLists.txt
|—— include
|   |—— mainwindow.h      //클래스 설계도
|—— resources
|—— src
|   |—— main.cpp          //진입점
|   |—— mainwindow.cpp    //구현
|—— ui
|   |—— mainwindow.ui      //버튼등 화면 배치
```

그리드는 4행 4열이고 숫자 버튼과 back, enter, shift, space 버튼이 배치되어 있다. 이전 과제처럼 좌표를 직접 찍지 않고 레이아웃을 썼기 때문에 창 크기를 바꾸면 버튼도 같이 늘어난다.

빌드 폴더에 output.txt가 만들어진다.

3. 코드 구조

헤더파일

private slots: on_MainWindow_iconSizeChanged가 있다. 이건 디자이너에서 자동으로 만들어진 것이라 실제로 쓰지는 않는다.

private: 실제 동작을 하는 함수들이다. key_press, commit, backspace, space, enter, toggle_shift, update_display가 있다. 이번에는 connect에 람다나 함수 포인터로 직접 연결했기 때문에 slots 부분에 넣지 않아도 동작한다.

그 아래에는 상태를 기억하는 변수들이 있다.

```
QTimer *timer;
```

```
QString text;
```

```

int last_key;    // 직전에 누른 키, 없으면 -1
int tap;         // 같은 키를 몇 번째 눌렀는지
int shift;      // 0:소문자 1:한글자만 2:고정
bool cur_upper; // 조합 중인 글자의 대소문자

```

이 프로젝트의 핵심은 이 변수들이다. last_key와 tap이 있어야 "같은 키를 또 눌렀는지"를 판단할 수 있고, 이게 없으면 그냥 누를 때마다 첫 글자만 나오게 된다.

Cpp

파일 맨 위에 키 배열이 있다.

```

static const char *keys[10] = {"0", ".?!", "abc", "def", "ghi", "jkl", "mno", "pqrs", "tuv",
"xyz"};

```

키 번호가 곧 배열의 인덱스가 되도록 순서를 맞춰 두었다. 이렇게 하면 3번 키를 눌렀을 때 keys[3]만 꺼내면 되어서 if문으로 하나씩 나눌 필요가 없다.

MainWindow::MainWindow(QWidget *parent) 생성자이다. 상태 변수들을 초기화하고 타이머를 만든다.

```

timer = new QTimer(this);
timer->setSingleShot(true);
connect(timer, &QTimer::timeout, this, &MainWindow::commit);

```

setSingleShot을 true로 하면 타이머가 한 번만 울리고 멈춘다. 이걸 안 하면 계속 반복해서 울리기 때문에 필요한 설정이다. 시간이 다 되면 commit이 실행되어 글자가 확정된다.

버튼 연결은 숫자 버튼 10개를 배열에 담아 반복문으로 처리하였다.

```

QPushButton *pad[10] = {ui->btn_0, ui->btn_1, ..., ui->btn_9};
for (int k = 0; k < 10; k++)
{
    connect(pad[k], &QPushButton::clicked, this, [=]{ key_press(k); });
}

```

clicked시그널은 어느 버튼이 눌렸는지 알려주지 않기 때문에, 람다로 감싸서 k를 같이 넘겨주는 방식을 썼다. 나머지 back, enter, space, shift 버튼은 각각 함수에 하나씩 연결하였다.

void MainWindow::key_press(int k) 숫자 버튼을 눌렀을 때 실행되는 함수이고 이 프로그램의 중심이다.

```
if (k == last_key && timer->isActive())    // 같은 키 연속 -> 다음 글자로 교체
{
    tap = (tap + 1) % set.length();
    text.chop(1);
}
else
{
    commit();
    tap = 0;
    last_key = k;
    cur_upper = (shift > 0);    // 조합 시작 시점에 대소문자 고정
}
```

두 가지 경우로 나뉜다. 직전에 누른 키와 같고 아직 타이머가 돌고 있으면 같은 글자를 바꾸는 것이므로, tap을 하나 올리고 방금 붙인 글자를 chop(1)로 지운 뒤 새 글자를 붙인다. tap에 나머지 연산을 쓴 이유는 마지막 글자에서 한 번 더 누르면 다시 첫 글자로 돌아가게 하기 위해서이다.

그게 아니면 새 글자를 시작하는 것이므로 commit으로 이전 조합을 끝내고 tap을 0으로 되돌린다. 이때 cur_upper에 지금 shift 상태를 저장해 두는데, 조합 도중에 shift가 풀려도 이미 시작한 글자의 대소문자는 안 바뀌게 하기 위해서이다.

마지막에 타이머를 700ms로 다시 시작한다. 여기서 restart가 되기 때문에 빠르게 연속으로 누르면 계속 같은 글자를 바꿀 수 있고, 손을 떼고 기다리면 확정된다.

void MainWindow::commit() 조합을 끝내는 함수이다. 타이머를 멈추고 last_key를 -1로, tap을 0으로 되돌린다. last_key가 -1이 되면 다음에 어떤 키를 눌러도 위의 if 조건에 걸리지 않기 때문에 새 글자로 시작된다. 타이머가 시간이 다 되어 부를 수도 있고, backspace나 space 같은 다른 버튼에서 부를 수도 있다.

void MainWindow::backspace() 먼저 commit으로 조합을 끝내고 chop(1)로 마지막 글자를 지운다. 조합 중에 지우면 상태가 꼬이기 때문에 commit을 먼저 부르는 것이 중요하다.

void MainWindow::space() 마찬가지로 commit을 하고 공백을 붙인다.

void MainWindow::enter() 입력한 문장을 파일에 저장하는 함수이다. 비어 있으면 return으로 즉시 종료한다.

```

QFile f("output.txt");
if (f.open(QIODevice::Append | QIODevice::Text))    // Append 라서 마지막 줄에 추가됨
{
    QTextStream out(&f);
    out << text << "\n";
    f.close();
}

```

Append 모드로 열었기 때문에 기존 내용을 지우지 않고 뒤에 계속 쌓인다. 저장이 끝나면 text를 비우고 화면을 갱신한다.

void MainWindow::toggle_shift() shift 값을 0, 1, 2로 돌려가며 바꾸고 버튼의 글씨도 같이 바꾼다. 0은 소문자, 1은 한 글자만 대문자, 2는 대문자 고정이다. 1일 때는 key_press 안에서 글자를 하나 찍고 나면 바로 0으로 돌아간다.

void MainWindow::update_display() lineEdit에 현재 text를 적는 함수이다. 앞의 함수들이 전부 마지막에 이 함수를 부른다. Day2의 refresh_view와 같은 역할이다.

4. 실행 결과

첨부된 영상 참고

5. 어려웠던 점

가장 어려웠던 것은 같은 버튼인데 상황에 따라 다르게 동작해야 한다는 점이였다. 처음에는 버튼을 누르면 글자를 붙이는 것만 생각했는데, 그러면 abc를 치려고 2번을 세 번 눌러도 aaa가 나온다. 직전 상태를 기억하는 변수가 필요하다는 걸 알기까지 시간이 걸렸다.

타이머를 언제 멈추고 언제 다시 시작해야 하는지도 헷갈렸다. 특히 backspace나 space를 눌렀을 때 commit을 안 부르면 그 뒤에 숫자를 눌렀을 때 엉뚱하게 앞 글자가 지워지는 문제가 있었다. 조합을 끝내는 동작을 commit이라는 함수 하나로 모아두고 필요한 곳마다 부르는 식으로 정리하니 해결되었다.

대소문자 처리도 예상보다 까다로웠다. shift가 1일 때 글자를 찍으면 바로 풀리게 했는데,

그 상태에서 같은 키를 한 번 더 눌러 글자를 바꾸면 이미 shift가 풀려 있어서 소문자가 되어 버렸다. 그래서 조합을 시작하는 시점의 shift 상태를 `cur_upper`에 따로 저장해 두는 방식으로 바꿨다.